Лабораторная работа 2. Фрагменты (Fragment vs Composable-функции) и их жизненный цикл

Необходимо разработать мобильное приложение под операционную систему Android, которое демонстрирует авторизованному пользователю набор данных (в соответствии с выбранной предметной областью), получаемых через интернет соединение (и сохраняемых в резервное хранилище для последующего использования вне интернет доступа).

1. Постановка задачи

В рамках данной лабораторной работы необходимо преобразовать каждую из существующих деятельностей (activity) в отдельные экраны (фрагменты или Composable-функции), настроить их взаимодействие внутри единой деятельности (MainActivity). Добавить логирование жизненного цикла как фрагментов (или Composable-функций), так и оставшейся единой деятельности.

1.1. Создание фрагментов экранов пользовательского интерфейса

Необходимо преобразовать 4 экрана пользователя в созвучные фрагменты (Composable-функции):

1. StartActivity → StartFragment
2. RegistrationActivity → RegistrationFragment
3. ContentActivity → ContentFragment
4. SettingsActivity → SettingsFragment (+резервное хранение)

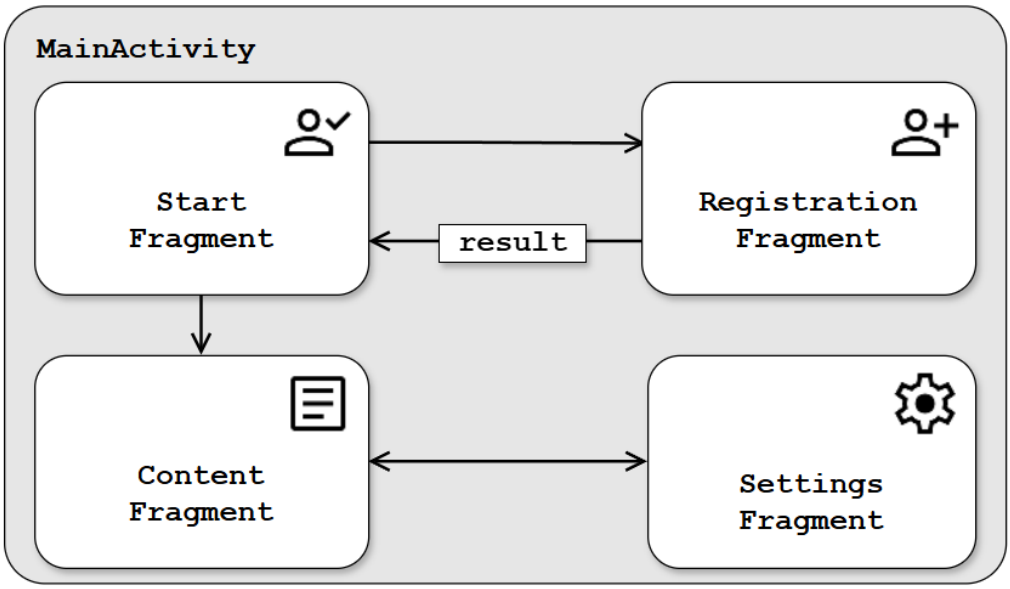
Оберните в комментарии код, отвечающий за переходы между экранами (деятельностями) предыдущей лабораторной работы. Перенесите код пользовательского интерфейса и логику из деятельности в соответствующие фрагменты (Composable-функции).

1.2. Создание единой деятельности

Создайте одну основную деятельность (MainActivity) для хостинга всех созданных фрагментов экранов:

- Убедитесь, что файл AndroidManifest.xml содержит только данную деятельность и что она является «стартовой точкой»;
- Реализуйте переходы между фрагментами (экранами), используя подходящий механизм для выбранного вами стека разработки.

При этом навигация переходов между экранами должна соответствовать предыдущему сценарию:



Внимание, если пользователь дошел до окна настроек, то по кнопке «назад» он должен видеть не более чем 1-2 предыдущих экранов. Не забывайте закрывать неактуальные экраны.

1.3. Логирование жизненного цикла экранов

Необходимо убедится, что осталось и функционирует логирование жизненного цикла единственной деятельности в соответствии с заданием предыдущей лабораторной работы.

Кроме этого, каждый из экранов (фрагментов или Composable-функций) должны выводить сообщение о переходе к иному состоянию своего жизненного цикла, используя подходящий механизм для выбранного вами стека разработки.

Необходимо предоставить стек (с фильтром на персональные логи), полученных вызовов для сценария перехода через все 4 окна приложения (стартовое, регистрация, данные, настройки и выход из приложения).

2. Организация логирования

2.1. Для проектов с XML верстkой

Каждый из фрагментов приложения (самостоятельно или за счет родительского класса) должны выводить во встроенную систему логирования тестируемых приложений (LogCat) уведомление о запуске каждого из методов жизненного цикла фрагментов (developer.android.com/fragments) в формате:

```
[название фрагмента]: [название метода]
```

Например:

```
StartFragment: onViewStateRestored()
```

2.2. Для проектов с Jetpack Compose

Каждый из экранов приложения должен выводить во встроенную систему логирования тестируемых приложений (LogCat) уведомление о запуске каждого из методов жизненного цикла (developer.android.com/androidx) в формате:

```
[название экрана]: [название события]
```

Например:

```
StartFragment: ON_START
```

По аналогии с фрагментами отслеживание жизненного цикла экранов (https://developer.android.com/topic/libraries/architecture/lifecycle) в Jetpack Compose позволяет останавливать и перезапускать трудоемкие задачи, подписываться и отказываться от различных служб и многое другое.

3. Организация переходов между фрагментами

3.1. Для проектов с XML верстkой

1. Подключите инструмент View Binding для автоматической проверки типобезопасности:

- Включите View Binding в файле build.gradle (Module) вашего приложения. Убедитесь, что ваш проект успешно собирается после включения View Binding.
- Замените текущий код работы с пользовательским интерфейсом на использование View Binding в оставшейся деятельности, во всех фрагментах или иных классах, использовавших метод findViewById.

2. В разметку (layout) MainActivity добавьте контейнер FragmentContainerView для отображения текущего активного фрагмента. Можно указать также стартовый фрагмент (по умолчанию) через его атрибут android:name;
3. В MainActivity создайте методы для перехода между различными фрагментами (например, navigateToRegistration,

navigateToContent и другие) и замените прямые вызовы перехода между деятельностями на вызовы соответствующих методов.

4. Используйте любой из механизмов для организации навигации между фрагментами на ваш выбор: FragmentManager (2018, устаревший) или навигационный граф (2019, Navigation Component)

3.1.1 Реализация навигации с использованием FragmentManager (https://developer.android.com/guide/fragments/fragmentmanager, **@Deplecated in API level 28**):

Реализуйте методы для перехода между различными фрагментами используя FragmentManager и FragmentTransaction

- Преобразуйте передачу данных между Activity в передачу данных между Fragment: https://developer.android.com/guide/fragments/create#add-programmatic

3.1.2. Использование Navigation Component для навигации между фрагментами (https://developer.android.com/guide/navigation/navigation-navigate, см. примеры для Views/Fragments):

- Добавьте зависимости для Navigation Component в файл build.gradle (Module) вашего приложения.
- Создайте файл навигации (navigation graph) в ресурсах проекта, где будут описаны все фрагменты и переходы между ними: https://developer.android.com/guide/navigation/use-graph/navoptions#options-xml;

  Реализуйте методы для перехода между различными фрагментами используя NavController и переходы, определенные в файле навигации;
- Преобразуйте передачу данных между фрагментами на использование инструмента Safe Args: https://developer.android.com/guide/navigation/navigation-pass-data

3.2. Для проектов с Jetpack Compose

1. В MainActivity используйте любой из механизмов для организации навигационного графа для экранов на ваш выбор: Navigation Component (2019, устаревший) или Navigation 3 (2025);
2. Преобразуйте передачу данных между Activity в передачу аргументов экранов (или изменение состояний компонентов)

3.2.1 Реализация навигации с использованием Navigation Component (https://developer.android.com/guide/navigation/navigation-navigate, см. примеры для Compose):

- Подключите актуальную версию библиотеки в файл build.gradle (Module) вашего приложения;
- Создайте навигационный граф внутри основной деятельности и реализуйте методы для перехода между различными экранами: https://developer.android.com/guide/navigation/design#compose;

3.2.2 Реализация навигации с использованием Navigation 3 (https://github.com/android/skills/tree/main/navigation/navigation-3/references/android/guide/navigation/navigation-3):

- Подключите актуальную версию библиотеки в файл build.gradle (Module) вашего приложения;
- Создайте навигационный граф внутри основной деятельности и реализуйте методы для перехода между различными экранами: https://developer.android.com/guide/navigation/design#compose.

3. Вспомогательные материалы

Основные источники информации:

- За информацией о жизненном цикле фрагментов в Android-приложениях стоит обращаться к сайту developer.android.com.
- На русском языке базовая информация представлена на сайте metanit.com: Kotlin+JetpackCompose или Java+XML

Список демо-проектов:

1. github.com/LenaDutova/Mobile-Android-App-Java/ проект на языке JAVA с XML верстkой (tree/example-2 или tree/example-3).
2. github.com/LenaDutova/Mobile-Android-App-Kotlin проект на языке Kotlin с XML верстkой (tree/example-2 или tree/example-3);

3. https://github.com/LenaDutova/Mobile-Android-App-Compose/tree/example-1 проект на языке Kotlin с JetpackCompose.